

## 課題 5.1 円周率の計算

モンテカルロ法を用いて円周率を求めるプログラムを書き，図 5.6 のように試行回数の増加による精度向上を確認しなさい。

## 1. コード

変数 整数 n 繰り返し回数及び生成された点の回数

実数  $x, y$     乱数で生成された  $(x, y)$

実数 pi モンテカルロで計算された円周率

整数 count  $|(x,y)| \leq 1$  である点の数

整数  $k$      10, 100, 1000, ... で出力するための制御変数

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

int main(int argc, char** argv){
    int n;
    double x, y;
    double pi;
    int count = 0;
    int k = 1;

    printf("n\t\tpi\t\t|pi - M_PI|\n");
    for(n = 1; n <= 1e9; n++) {
        x = ((double) rand()) / RAND_MAX;
        y = ((double) rand()) / RAND_MAX;
        if((x * x + y * y) <= 1) count += 1;
        pi = 4 * ((double) count) / n;
        if(n % k == 0) {
            printf("%d\t%.10f\t%.10f\n", 10*n, pi, fabs(pi - M_PI));
            k *= 10;
        }
    }

    return 0;
}
```

## 2. 実行結果

| n          | pi           | pi - M_PI |
|------------|--------------|-----------|
| 1          | 4.0000000000 | 0.858407  |
| 10         | 3.6000000000 | 0.458407  |
| 100        | 3.2000000000 | 0.058407  |
| 1000       | 3.1120000000 | 0.029593  |
| 10000      | 3.1140000000 | 0.027593  |
| 100000     | 3.1374000000 | 0.004193  |
| 1000000    | 3.1390880000 | 0.002505  |
| 10000000   | 3.1409324000 | 0.000660  |
| 100000000  | 3.1413526800 | 0.000240  |
| 1000000000 | 3.1415206120 | 0.000072  |

## 3. 実行結果の可視化

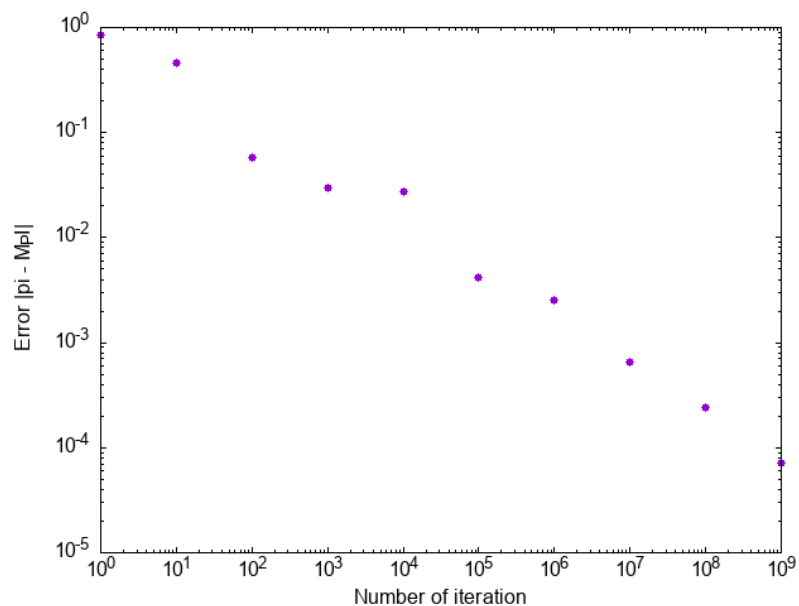


図 1.1 モンテカルロによる誤差の反復回数特性

反復回数が増えるとモンテカルロによる誤差が減少していくことが確認できた。回まで点を  $10^9$  回打った後、誤差が  $10^{-4}$  に収まった。両対数グラフで誤差の反復回数特性を可視化すると直線に近くなる。

## 課題 5.2 2次元ラプラスの方程式の数値解

2次元ラプラスの方程式の数値解を求めるプログラムを書きなさい。ここで、領域  $N_x \times N_y$  は  $100 \times 100$  (C言語では  $0 \sim 99$  までの  $100$ )、領域境界は  $y = N_y - 1$  の一辺を  $V(x, N_y - 1) = 100 \sin(\pi x / N_y)$  (他は  $0$ )、領域中の初期電位は  $V^0(x, y) = 0$ 、 $\varepsilon = 10^{-8}$  としなさい。結果は、図 7 のように電位の空間分布  $V(x, y)$  をプロットするとともに、反復回数の増加により解が収束することを確認しなさい。

### 1. コード

|    |    |           |                        |
|----|----|-----------|------------------------|
| 変数 | 実数 | error     | 誤差                     |
|    | 整数 | iter      | 反復回数                   |
|    | 実数 | $V[N][N]$ | 領域 $N_x \times N_y$ の点 |
|    | 整数 | i,j,k     | 制御変数                   |

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define N 100

int main(int argc, char **argv){
    double error = 1e-8;
    int iter = 100000;
    double V[N][N] = {0};
    int i, j, k;

    FILE *output;

    for(i = 0; i < N - 1; i++){
        V[i][0] = V[0][i] = V[N - 1][i] = 0;
    }

    for(i = 0; i < N - 1; i++){
        V[i][N - 1] = 100 * sin(M_PI * i / N);
    }

    output = fopen("./5.2.shusoku.tsv", "w+");
    for(k = 0; k < iter; k++){
        for(i = 1; i < N - 1; i++){
            for(j = 1; j < N - 1; j++){
                V[i][j] = (V[i + 1][j] + V[i - 1][j] + V[i][j + 1] + V[i][j - 1]) / 4;
            }
        }
    }
}
```

```

        fprintf(output, "%d\t%.10f\n", k, V[N/2][N/2]);
    }

    fclose(output);

    output = fopen("./5.2.result.tsv", "w+");
    for(i = 0; i < N; i++)
        for(j = 0; j < N; j++){
            fprintf(output, "%2d\t%2d\t%.10f\n", i, j, V[i][j]);
        }
    fclose(output);

    return 0;
}

```

最初領域境界  $V(x, N_y - 1) = 100 \sin\left(\frac{\pi x}{N_y}\right)$  とし、その他は 0 とする。それから  $k = 1$  から  $k = iter$  まで各点における電位を計算し、 $V\left[\frac{N}{2}\right]\left[\frac{N}{2}\right]$  を 5.2.shusoku.tsv に書き書き込む。その後、すべての点の電位を 5.2.result.tsv に書き込む。

## 2. 実行結果の可視化

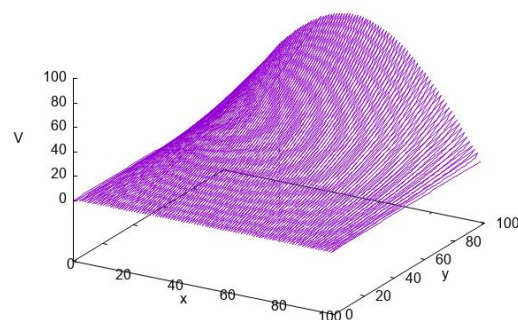


図 2.1 電位の空間分布

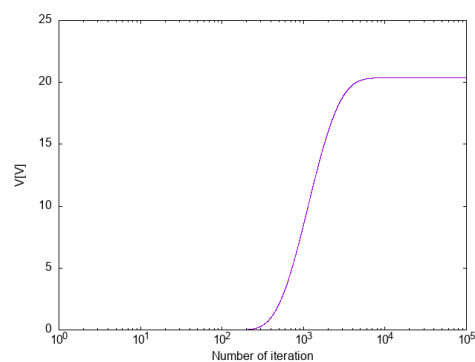


図 2.2 反復回数による解の収束  $V[50][50]$

図 2.2 を見ると反復回数  $10^5$  まで実行すると  $V[50][50]$  が約 20 [V] に収束した。反復回数 200 までは変化が小さかったが、そこから反復回数 3000 まで電位が急に変化し、徐々に収束した。

## 考察

モンテカルロ法による円周率近似の精度を考察する。正方形  $1 \times 1$  内の四分の一の単位円に点が打たれる確率  $p$  は  $\frac{\pi}{4} \approx 0.785398$  である。すべての点数を  $n$ 、円弧内に落ちる点数を  $x$  とし、 $x$  の確率分布は期待値が  $\mu = np = \frac{n\pi}{4}$  で分散が  $\sigma^2 = np(1-p)$  の二項分布になる。 $n$  を大きくすればこの二項分布を正規分布とみなせるから、99.7%の乱数が円弧の中に入る範囲を見積もることができる。正規分布の性質から 99.7%の乱数が円弧の中に入る範囲は  $[\mu - 3\sigma, \mu + 3\sigma]$  である。

分布の中央を  $\pi$  にしてモンテカルロ法による近似値が 99.7%の確率で入る範囲計算する。 $\mu = \frac{n\pi}{4}$  から新しい中心  $\mu'$  は  $\frac{4}{n}$  をもとの確率変数に掛けたものである。 $\mu' = \pi = \frac{4\mu}{n}$  である。分散の性質  $V[aX] = a^2V[X] = a^2\sigma^2$  により、新しい標準偏差は  $\sigma' = \frac{4}{n}\sigma$  になる。

$$\begin{aligned}\mu' \pm 3\sigma' &= \pi \pm \frac{4}{n} \cdot 3\sqrt{np(1-p)} \\ &= \pi \pm \frac{3\sqrt{\pi(1-\pi)}}{\sqrt{n}} \\ &\approx \pi \pm \frac{4.926550}{\sqrt{n}}\end{aligned}$$

モンテカルロ法による円周率の近似値は  $\left[\pi - \frac{4.926550}{\sqrt{n}}, \pi + \frac{4.926550}{\sqrt{n}}\right]$  の範囲に 99.7%の確率で集まることになる。すなわち、誤差は 99.7%の確率で  $\frac{4.926550}{\sqrt{n}}$  以下になる。

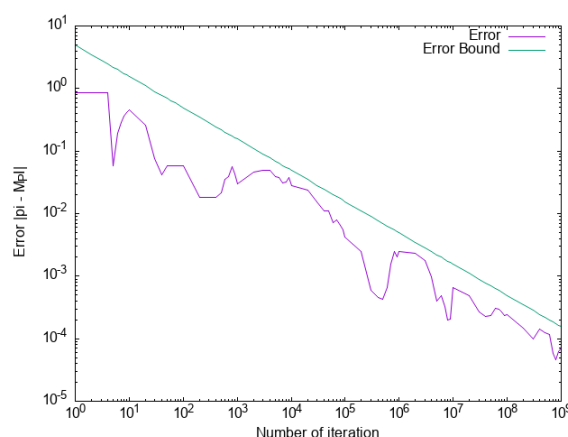


図 3.1 モンテカルロによる誤差の反復回数特性

図 3.1 はモンテカルロ法による誤差と確率 99.7%で誤差が超えない上限を示す。今回の実験ではその境界を越えた誤差がなかった。